AUTOMATED WEB SECURITY SCANNER USING PYTHON

Cybersecurity Project Report

Submitted by: [Your Name]

Course / Branch: [Your Course]

College: [Your College]

Submission date: [Insert Date]

Please replace the bracketed details and add your own screenshots and actual test results before submission.

1. Abstract

The Automated Web Security Scanner is a Python-based educational tool that checks a website for selected HTTP security configuration indicators. It examines whether HTTPS is used and whether six commonly discussed response headers are present: Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy. The program prints a summary and saves findings in JSON and CSV formats. For each missing header, it provides a short explanation and a practical recommendation. The project demonstrates how Python can automate basic defensive security checks and produce repeatable documentation. It is intended for authorized, read-only assessment and learning. It is not a full vulnerability scanner, does not exploit systems, and cannot guarantee that a website is secure.

2. Problem Statement

Website security configuration can be difficult for beginners to review manually. Important HTTP response headers may be missing or misunderstood. A small automated tool can help learners identify selected configuration indicators and understand why they matter.

3. Objectives

- Automate checks for HTTPS usage and selected security headers.
- Explain the purpose of each check in plain language.
- Suggest remediation steps for missing headers.
- Export findings to JSON and CSV for review.
- Demonstrate safe, read-only cybersecurity testing.

4. Proposed Solution

The user supplies a URL. The program validates the URL, performs a single HTTP GET request without following redirects, inspects the response status and headers, evaluates the selected checks, calculates a simple educational score, and writes a JSON report and CSV table. Network errors are reported instead of being treated as successful scans.

5. System Architecture

User enters URL → URL and hostname validation → Read-only HTTP request → Response/header inspection → Rule-based checks → Terminal summary → JSON and CSV reports.

6. Tools and Technologies

- Python 3: application logic and report generation.
- Requests: HTTP request handling and timeouts.
- JSON: structured report output.
- CSV: spreadsheet-compatible findings.
- Visual Studio Code: recommended editor and terminal.

7. Methodology

The scanner checks HTTPS usage and the presence of six headers. It records whether each header exists and prints its purpose. Missing headers are accompanied by suggested mitigations. A simple score counts the HTTPS check and each present header. This score is for demonstration only and is not a standardized security rating.

8. Implementation and Run Commands

Windows PowerShell:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

```
python scanner.py https://example.com
```

The example target is for demonstration; use only a website you own or have permission to assess. If PowerShell activation is blocked, use Command Prompt and run `.venv\Scripts\activate.bat`.

9. Testing Plan

Test a reachable permitted HTTPS target, a target with missing headers, an invalid hostname, a timeout case where available, an HTTP endpoint, and a URL that redirects. The included tests/TEST_PLAN.md gives the expected behavior. Add real test dates, outputs, and screenshots after you run the program; do not present hypothetical results as actual results.

10. Advantages

• Beginner-friendly command-line interface.
• Fast, repeatable checks.
• Clear recommendations for missing headers.
• JSON and CSV output for documentation.
• Read-only and does not attempt exploitation.

11. Limitations

The scanner checks only selected headers and HTTPS URL usage. Header presence does not establish that a policy is correct. It does not test application logic, authentication, SQL injection, cross-site scripting, or server-side vulnerabilities. The score is illustrative. Redirects are not followed. DNS and network behavior can be complex; the basic hostname checks are not a complete defense against all network-routing risks.

12. Future Enhancements

Possible improvements include a dashboard, scheduled scans, configurable header policies, richer report formatting, unit tests, and integration into a CI pipeline. Any extension should retain authorization checks and safe request limits.

13. Conclusion

This project demonstrates a practical use of Python for basic defensive web security configuration checks. It helps learners understand selected security headers, review findings, and document recommendations. It should be treated as a learning aid rather than a substitute for a professional security assessment.

14. References

• OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/
• OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
• Requests documentation: https://requests.readthedocs.io/en/latest/
• Python documentation: https://docs.python.org/3/

15. Evidence to Add Before Submission

Add screenshots of: (1) the project files in VS Code, (2) dependency installation, (3) a real scan you are authorized to perform, (4) the generated JSON report, and (5) the CSV report. Fill in your personal/course details on the cover page. Report only tests you actually ran.

Appendix A: Viva Preparation

Q1. What is the purpose of this project?
It checks selected web security headers and HTTPS usage, then reports findings.

Q2. What is CSP?
Content Security Policy helps control the sources from which a browser may load resources.

Q3. Why use HTTPS?
HTTPS encrypts traffic in transit when configured correctly.

Q4. What does the score mean?
It is a simple educational count of passed checks, not a complete security rating.

Q5. Does this prove a website is secure?
No. It checks only selected configuration indicators and is not a full vulnerability assessment.